

AWS Project Report

Albert Huang 530215563 qhua0468

1 Introduction

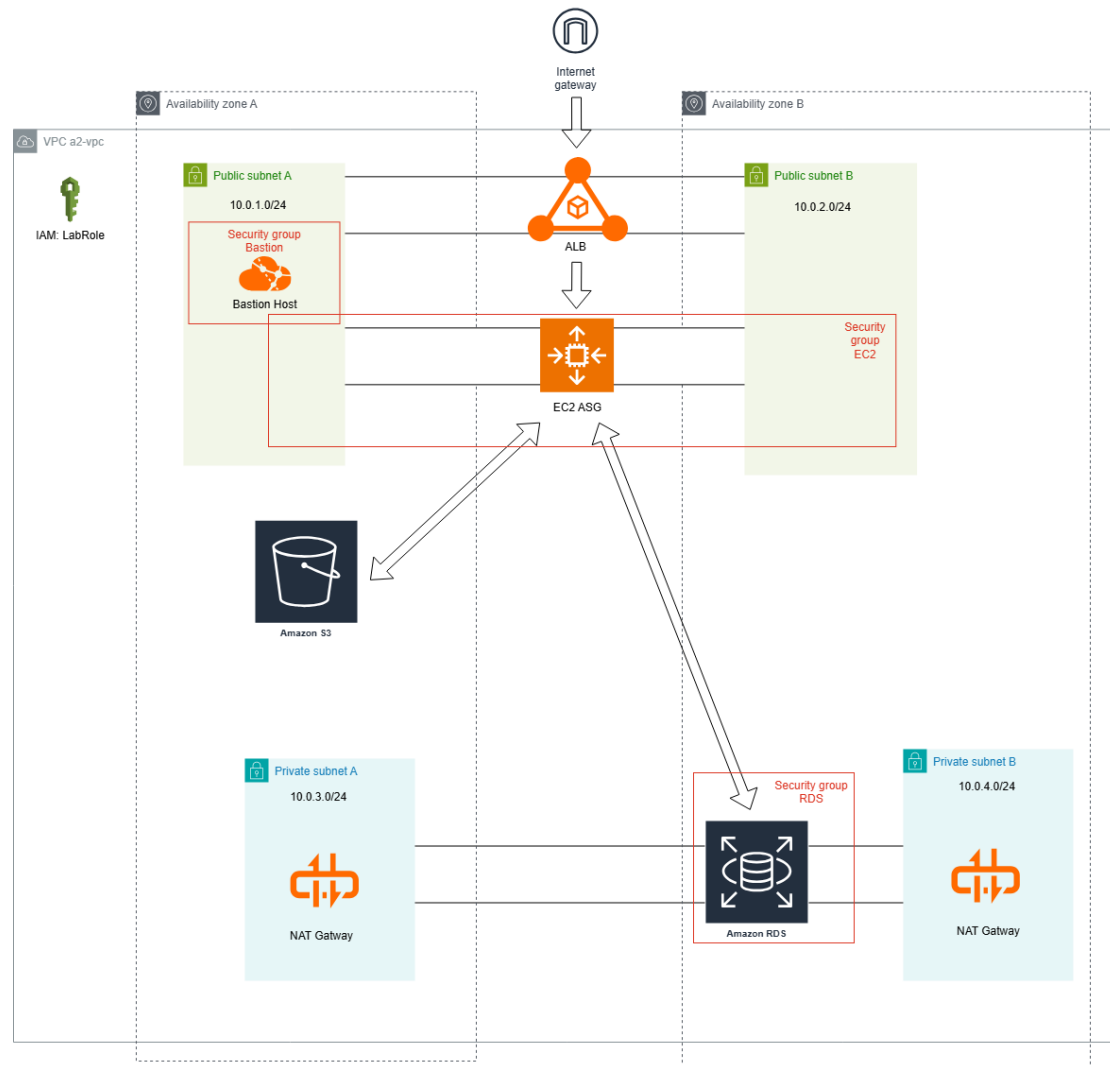
This is AWS deployment project inherits the foundation of deploying a web app from the previous AWS deployment exercise. It aims to deploy a web application that can generate caption and thumbnail from an upload image. Different from assignment 1, this time the project requires the deployment of serverless methods, specifically through AWS Lambda. Other components like Auto Scaling Group (ASG), Auto Load Balancer (ALB) etc. are key in this project. The development of this project covers multiple topics throughout the unit, including VPC network environment, database in Amazon RDS, scalable and highly available environment, automating infrastructure deployment, serverless architecture and more.

A successful outcome of this project should guarantee the mastery of AWS architecture and enhance the experience and skills in solving a web application deployment problem that reflects on the real-world scenarios.

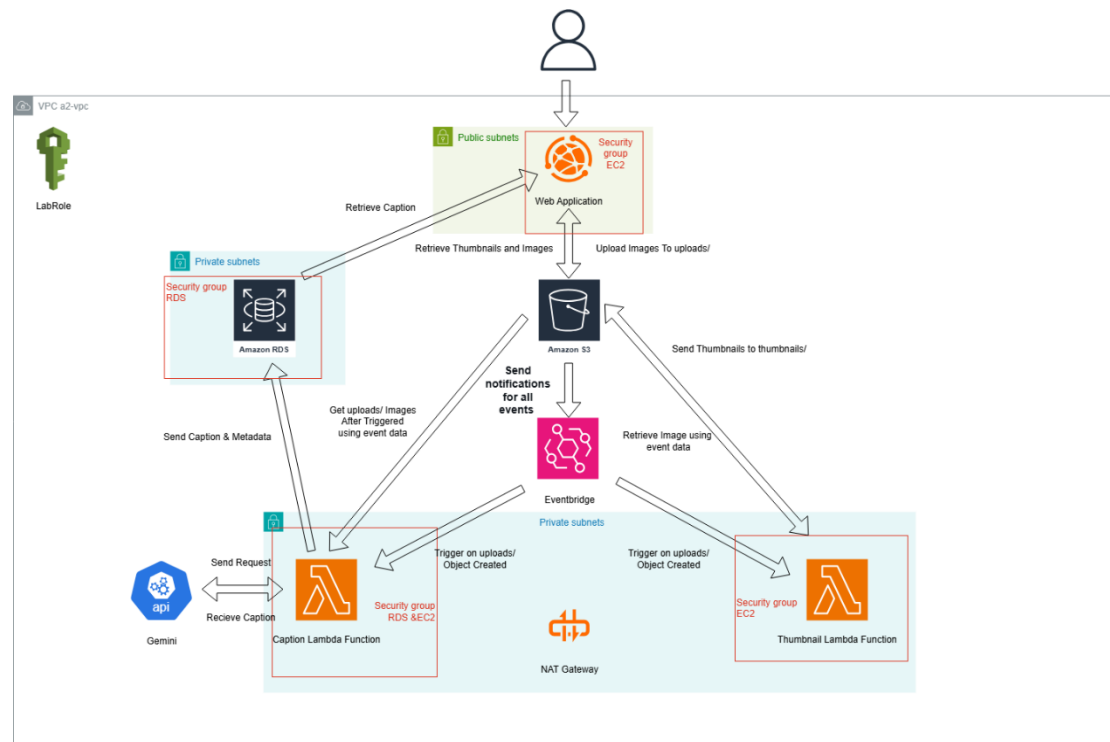
This report will showcase the learn and gain from this project by demonstrating architecture diagrams, explaining web application deployment, serverless component deployment, giving auto scaling test observation, and finally conclude the project with a summary and discussion of learn.

2 Architecture Diagram

2.1 Web Application Architecture



2.2 Serverless Architecture



2.3 Interaction Between Components

As we can see, there are two architectures that support this project. One is Web application architecture, and the other is serverless architecture. And these two architectures' components interact with each other to perform the functions we desire.

One main interaction is between the web application and the Lambda functions. The web application is hosted on a group of EC2 instance (ASG) and the user accesses the web application through DNS name given from load balancer. The user will upload images using the web application, then the web application will store these images in the "uploads/" folder. The S3 will send notifications to eventbridge for all the events happening in S3. The eventbridge will follow a created rule which will trigger the two lambda functions on the object creation event in "uploads/" folder.

Another main interaction about the components from both sides access their share resources on S3 and RDS. First, the caption lambda function sends the generated captions along with the metadata of associated images, then the web application retrieves these captions along with the metadata (image key). Second, the thumbnail lambda function sends the generated thumbnails to the "thumbnails/" folder in the S3, then the web application will retrieve the thumbnails with the corresponding original images using the image key gained from the RDS.

3 Web Application Deployment

3.1 Compute Environment

The frontend of this project runs on EC2 instances that managed by ASG I created to ensure high availability along with elastic capacity. All the components are using IAM **LabRole** when applicable and using the **vpc** I customized called “a2-vpc” (10.0.0.0/16). In that vpc, there are 4 subnets:

Two **public subnets** (10.0.1.0/24 and 10.0.4.0/24) in different available zone (us-east-1a and us-east-1b). These public subnets are in route table called “a2-public-rt” and eventually to the internet gateway “a2-igw”.

Two **private subnets** (10.0.2.0/24 and 10.0.3.0/24) in different available zone (us-east-1a and us-east-1b). These public subnets are in route table called “a2-private-rt” and connect to a NAT gateway.



For the **EC2 instances**, they are in all the same specifics using launch templates. In that template, I selected Amazon Linux 2023 as the AMI and t2.micro as instance type. Key pair is using vockey as required. The security group we selected is ec2-sg which will be explained in detail later. I used user data to download the packages and dependencies needed from the flask application to run. The script will download and unzip the deployment folder containing the flask application code and html files store in S3. These files are stored in a separate folder and run the flask application.

User data



```
#!/bin/bash
yum update -y
yum install -y python3 git
sudo yum install python3-pip -y
pip install flask
pip install mysql-connector-python
pip install boto3 werkzeug

cd /home/ec2-user
aws s3 cp s3://a2-image-bucket-530215563/deployment/app.py app.py
aws s3 cp s3://a2-image-bucket-530215563/deployment/templates/ templates/ --recursive

python3 app.py
```

For security configurations, I set three **security groups** here. They are ec2-sg, bastion-

sg and rds-sg. First, the ec2-sg allows SSH from bastion, http from anywhere in port 80 for web application and tcp in port 5000 for RDS, with outbound rule all traffic to anywhere. Then the bastion-sg only allows SSH from my IP address to ensure security and outbound rule as the same as above. Lastly, rds-sg allows TCP 3306 which is MySQL/Aurora from source ec2-sg and bastion-sg meaning the EC2 instance and bastion host to ensure security.

For **administrative access**, I used a bastion host here. The bastion host is using its own bastion-sg security group that only allow SSH from my IP address. I can use SSH through bastion to admin EC2 instance or RDS with the vokey given. This satisfies the minimization of exposure and performance a secure practice.

For **load balancer**, it is being selected and attached to the AGS. It is listening on http 80 then forwards to target group called "a2-tg" which registered all EC2 instances in the ASG and check their healthy status. The path of health checking is set to / with http, 2 healthy threshold, 5 second timeout and 15 seconds interval. By doing so, as the ALB is deployed in both available zones (us-east-1a and us-east-1b), with the ASG required at least one instance in each available zones, this ensures route-balances across healthy targets in multiple available zones. When one instance is terminated or offline for some reasons, the ALB will just stop that traffic and reroute to a healthy one.

3.2 Database Environment

For **RDS**, I used instance as role, MySQL Community as engine with db.t3.micro as class. It used its own security group rds-sg and two private subnets to ensure the access is only MySQL from bastion host or EC instance that host the web application and ensure the connection to the caption lambda function. The database and table within were created by using a shell from AWS deployment exercise that create a database called image_caption_db and a table called captions that have structures to store the metadata and caption of images. I access the RDS through bastion host with SSH vokey and execute this shell command for database creation. As for integration, the flask application doesn't change the content in RDS. The application only try connect to RDS and retrieve image keys and captions form the table.

```
try:
    connection = get_db_connection()
    if connection is None:
        return render_template("gallery.html", error="Database Error: Unable to connect to the database.")

    cursor = connection.cursor(dictionary=True)
    cursor.execute("SELECT image_key, caption FROM captions ORDER BY uploaded_at DESC")
    results = cursor.fetchall()
    connection.close()
```

The caption lambda function however, connect to RDS and insert (image_key, caption) into the table for each event (each new uploads/ image).

3.3 Storage Environment

In this project, I used a general purpose **S3** bucket. It has a "deployment/" folder that holds the flask application python file and templates of the website, waiting to be downloaded by each EC2 instance. It also has a folder "uploads/" where all the uploaded original images are stored. And it has a folder "thumbnails/" where all the generated thumbnails are stored.

The “Send notifications to Amazon EventBridge for all events in this bucket” option is enabled so that it can send notification to eventbridge for further filtering then trigger the two lambda functions.

4 Serverless Component Deployment

4.1 Event-Driven Architecture

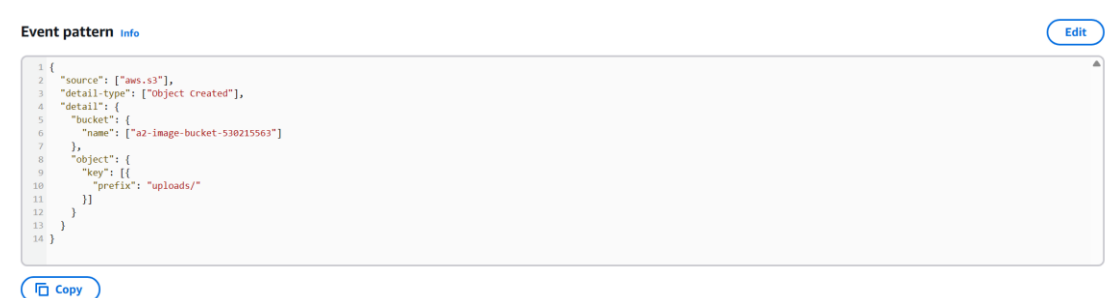
In the section 2.3, I have briefly introduced the triggering of lambda functions. In section, we will explain the event-driven architecture in more details.

To begin with, the choice of event-driven architecture is very important. It is vital to understand that this requires the trigger of two separate lambda functions for a single event. Meaning that after one image is uploaded into S3, the two lambda functions should be triggered by this at the same time. Thus, eventbridge is the perfect choice for this scenario.

Then, to avoid the recurring trigger of the lambda function as the thumbnail lambda function also write thumbnails into S3, we created two folders, “uploads/” and “thumbnails/” to store the images and their thumbnails separately.

For eventbridge to receive notifications that includes the packages we need, we enable the option in S3 properties to let it send notifications to eventbridge for all events in this bucket.

Next, I created a rule in eventbridge for the correct trigger of lambda functions. I specific the type of event as ObjectCreated in S3 with the prefix “uploads/”. This will filter out the events in thumbnails to avoid recurring issues.



After I set the event pattern, I selected both lambda functions as the target. This is the whole workflow of creating lambda functions trigger.

When the lambda functions are triggered. They interact with various resources. For the caption/annotation lambda function, it first retrieves the uploaded image in the event from S3 using the image key and bucket name in event data. Then it calls Gemini API using my google API key, sent the encoded version of the image to the API and request for a caption. After it received the caption generated, it then sends the caption along with the corresponding metadata of that image (image key) to the image_caption_db in the RDS. For the thumbnail lambda function, it first retrieves the uploaded image in the event from S3 the same way as caption/annotation lambda function. Then, it generates the corresponding thumbnails using PIL dependencies. The thumbnails are sent to S3's “thumbnail/” folder.

4.2 Annotation Function

First, the core functions are achieved in a single `lambda_function.py` file. In that file, I get the bucket name and the key name of the object from the eventbridge envelope. And developed double assurance of avoiding recurring by skipping any key with “thumbnails/” or not ending with image format.

```
def lambda_handler(event, context):
    print("Received event:", json.dumps(event))

    key = event.get('detail', {}).get('object', {}).get('key', '')
    bucket = event.get('detail', {}).get('bucket', {}).get('name', '')

    if key.startswith("thumbnails/") or key.startswith("captions/") or not key.lower().endswith(('.jpg', '.jpeg', '.png')):
        print("Skipped key:", key)
        return {"statusCode": 200, "body": "Skipped file."}
```

Then, I download the image using the information gained previously from S3. The image is being base64-encoded then and send to Gemini REST API endpoint through a POST using my Gemini API key. After the response from API is received, we parse the json result and extract the caption text. Then we connect the RDS and send the image key along with its caption to the captions table in `image_caption_db`. `pymysql.connect(...)` is used here for insertion.

The Gemini key API, credentials of the RDS are store in the environment variables by setting lambda function's configuration. We installed all dependencies needed in the same folder as the python file in Cloud9 environment. Then the folder is zipped and uploaded to the lambda function. We ensure the version of runtime matches the version of python used for resources in the zip file.

The timeout of this lambda function is set to 30 seconds since the default 3 seconds is too short for the operation in this function (Gemini and RDS). The environment variables are set are described above. And VPC is set to the custom vpc `a2-vpc` with the two private subnets. The security group is using both `ec2-sg` and `rds-sg`. The IAM role is using the given `LabRole`. By attaching the two private subnets and the two security groups. This function can call API using NAT gateway set in the two private subnets and connect to RDS instance correctly.

4.3 Thumbnail Generator

For thumbnail lambda function. I follow the similar method and process. First, I prepare a python file covering all the functions needed in a folder in cloud9 environment. In the python file, we import `boto3` and `PIL.Image`. A similar coding to get bucket name and object key from event envelope with double check the prefix and type of the object is adopted here. Then, I get the image from S3 using this information. `Image.open(io.BytesIO(image_data))` and `img.thumbnail((128,128))` are used to generate a thumbnail in 128 x 128 size. We then save the thumbnail in a buffer and use that buffer to send the thumbnail into the “thumbnails/” folder in S3.

The packaging is the same as the caption lambda function. I installed all the dependencies needed in the same folder as the python file then zipped the entire folder and

uploaded it in the lambda function console.

I also ensure the version of python of runtime is the same as in the resources in zip file. The vpc, subnets settings are the same as caption lambda function with a2-vpc and two private subnets. The security group here is only selected as ec2-sg as it does not need to connect to RDS, only S3. And the IAM role is still LabRole. The remain of configuration remains unchanged or as default.

5 Auto Scaling Test Observation

The attempts of auto scaling test here is using Apache benchmark. This tool only sends HTTP GET requests, and it will not trigger any lambda functions. The command `ab -n 1000000 -c 1000 / a2-alb-1656092265.us-east-1.elb.amazonaws.com /` is used. This will send 1000 request, continuously sending large amount of http GET. And with a certain amount of time the CPU is expected to be $\geq 60\%$ easily.

6 Summary and Lessons Learned

By completing this project, I have learnt some key findings and lessons by solving challenges. First, I learnt to debug the running of flask application by connecting to EC2 instance on AWS console. By manually checking the repository through commands like `ls` and other dependencies checking commands I can check whether the user data script in the template is being execute as expected. Adjustment can be made from there accordingly. This is the same for flask application by manually running the `app.py` file to see if there is any error. Second, I learnt to trigger multiple lambda functions without recurring issues by using eventbridge, prefix, S3 notification and setting good structure of storage. Third, runtime version needs to be matched with the version in dependencies and functions files. The timeout session should be adjusted accordingly for specific lambda functions. Fourth, endpoint access is a way to go when the dependencies are too big for a zip file in lambda function. Lastly, I leant the configuration of web, from vpc to subnet, routable, NAT gateway etc. with good practices.

In conclusion, the successful deploy of this project marks the increased mastery of me in web application and serverless deployment for AWS cloud computing.